



**UNIVERSIDAD NACIONAL DEL
NORDESTE**



**Facultad de Ciencias Exactas y Naturales y
Agrimensura**

Ingeniería de Software II

Trabajo de campo (TC)

Nombre proyecto: “Univia”

Grupo N.º 64

Integrantes:

- Acosta, Lara Diamela Itati
- Benitez, Valentina Itatí

ÍNDICE DE CONTENIDOS

Capítulo 1: Introducción

1.1 Introducción	6
1.2 Breve estado del arte	6
1.3 Objetivos	7
1.3.1 Objetivo general	7
1.3.2 Objetivos específicos	7
1.4 Fundamentación	7
1.5 Alcance	8

Capítulo 2: Metodología

2.1 Ciclo de vida del proyecto	9
2.2 Planificación realizada	9
2.2.1 Nuestros recursos humanos.....	9
2.2.2 Planificación del Sprint 1	10
2.2.3 Planificación del sprint 2	10
2.3 Cronograma del proyecto	10
2.3.1 Relación entre Sprints y funcionalidades	12
2.4 Plan de riesgos	13
2.4.1 Riesgos del producto	13
2.4.2 Riesgos del proyecto	13
2.4.3 Riesgos de negocio	14
2.5 Evaluación de arquitecturas	14
2.5.1 Selección de la arquitectura	14
2.5.2 Justificación	15
2.6 Diagrama de casos de uso	17

2.7 Conversaciones	17
2.8 Diagrama de secuencia	21
2.9 Contrato de operaciones críticas	23
2.9.1 Contrato: Guardar Publicación.....	23
2.9.2 Contrato: Búsqueda de material	24
2.10 Diagrama de entidad relación	25
2.11 Diccionario de datos	26

Capítulo 3: Herramientas

3.1 Herramientas utilizadas	34
3.2 Bibliografía consultada	34
3.3 Autoevaluación	34

Anexos

Anexo A. Enlace al repositorio git	36
Anexo B. Diagrama de secuencia anterior	36
Anexo C. Código fuente actualizado	36

ÍNDICE DE FIGURAS

Figura 1: Organización del proyecto mediante trello	12
Figura 2: Diagrama de arquitectura	16
Figura 3: Diagrama de casos de uso	17
Figura 4: Diagrama de secuencia: Buscar Material.....	22
Figura 5: Diagrama de secuencia: Obtener Material.....	22
Figura 6: Diagrama de secuencia: Subir Material.....	23
Figura 7: Diagrama de entidad relación	25
Figura 8: Diagrama de secuencia anterior	36

ÍNDICE DE TABLAS

Tabla 1: Planificación de actividades	11
Tabla 2: Caso de uso: Buscar material	18
Tabla 3: Caso de uso: Obtener material	19
Tabla 4: Caso de uso: Subir un nuevo material	21
Tabla 5: Contrato de operaciones críticas C.U. Obtener material	23
Tabla 6: Contrato de operaciones críticas C.U. Subir un nuevo material	24
Tabla 7: Diccionario: Pais	26
Tabla 8: Diccionario: Ciudad	26
Tabla 9: Diccionario: Facultad	27
Tabla 10: Diccionario: Universidad	28
Tabla 11: Diccionario: Carrera	29
Tabla 12: Diccionario: Carrera_materia	29
Tabla 13: Diccionario: Materia	30
Tabla 14: Diccionario: Usuario	31
Tabla 15: Diccionario: Publicacion	32
Tabla 16: Diccionario: Archivo	33

Capítulo 1: Introducción

El presente documento corresponde al primer avance del proyecto de software “Univia”, continuación del trabajo desarrollado en la asignatura Ingeniería de Software I. En esta etapa, el objetivo principal es avanzar desde el análisis inicial hacia la definición y desarrollo de aspectos más concretos del sistema.

Para este avance, se ha definido como alcance la implementación parcial de dos funcionalidades principales del sistema: **búsqueda de materiales académicos** y **publicación de nuevos materiales**. Estas funcionalidades fueron seleccionadas por ser fundamentales dentro del funcionamiento de la plataforma, ya que permiten tanto la generación como el acceso a los recursos compartidos por los usuarios.

A partir de este alcance, se desarrollan los elementos necesarios para su implementación, incluyendo la planificación del proyecto, el análisis de riesgos, la evaluación y selección de la arquitectura de software, y la elaboración de modelos como diagramas de casos de uso refinados, diagramas de secuencia y contratos de operaciones críticas.

Asimismo, se avanza en el desarrollo de una funcionalidad básica, con el fin de validar las decisiones de diseño adoptadas y comenzar a materializar el sistema propuesto.

De esta manera, este avance mantiene coherencia con las etapas previas del proyecto, permitiendo una evolución progresiva del mismo y aplicando los conceptos y buenas prácticas de la Ingeniería de Software.

1.2 Breve estado del arte

En la actualidad, existen diversas plataformas digitales que permiten el acceso y la distribución de material académico, tales como repositorios institucionales, bibliotecas virtuales y servicios en la nube como Google Drive. Estas herramientas facilitan el almacenamiento y la consulta de documentos, pero en muchos casos presentan limitaciones en cuanto a organización, búsqueda específica y confiabilidad del contenido.

Asimismo, muchos estudiantes recurren a redes sociales o grupos de mensajería, como WhatsApp o Telegram, para compartir apuntes y materiales de estudio. Sin embargo, estos medios suelen ser desorganizados, temporales y poco eficientes para encontrar información de manera rápida y precisa.

Por otro lado, algunas plataformas más estructuradas ofrecen funcionalidades de intercambio de recursos, pero no siempre están adaptadas al contexto local o a las necesidades específicas de los estudiantes universitarios, como la posibilidad de filtrar por carrera, materia o institución.

En este contexto, surge la necesidad de desarrollar una solución que integre estas funcionalidades en un entorno organizado, accesible y confiable, centrado en la colaboración entre estudiantes y en la optimización de la búsqueda y publicación de materiales académicos.

1.3 Objetivos

1.3.1 Objetivo general

Desarrollar una plataforma digital que facilite la búsqueda y publicación de materiales académicos entre estudiantes universitarios y terciarios, promoviendo el acceso al conocimiento y la colaboración entre usuarios.

1.3.2 Objetivos específicos

- Implementar una funcionalidad que permita a los usuarios buscar materiales académicos mediante filtros y palabras clave.
- Desarrollar una funcionalidad que permita a los usuarios publicar materiales académicos de forma sencilla y organizada.
- Garantizar una interacción intuitiva y accesible para los usuarios del sistema.
- Aplicar conceptos y herramientas de Ingeniería de Software en el desarrollo del proyecto.
- Validar el funcionamiento del sistema mediante la implementación de una funcionalidad básica.

1.4 Fundamentación

El desarrollo del sistema “Univia” surge a partir de una problemática frecuente en el ámbito académico: la dificultad de los estudiantes para acceder a materiales de estudio de forma rápida, organizada y confiable. En muchos casos, los recursos necesarios no se encuentran fácilmente disponibles en fuentes oficiales, lo que obliga a recurrir a medios informales como grupos de mensajería o contactos personales.

Esta situación genera pérdida de tiempo, desorganización y, en ocasiones, acceso a información incompleta o poco confiable. Frente a esto, resulta necesario contar con una plataforma que centralice la búsqueda y publicación de materiales académicos, permitiendo a los estudiantes compartir recursos de manera estructurada y eficiente.

Además, el sistema busca fomentar la colaboración entre usuarios, promoviendo una comunidad basada en el intercambio de conocimiento y la ayuda mutua. Desde el punto de vista académico, este proyecto permite aplicar y consolidar los conocimientos

adquiridos en Ingeniería de Software, abordando tanto aspectos de análisis como de diseño e implementación.

1.5 Alcance del proyecto

En el presente avance, el alcance del proyecto se centra en la implementación parcial de dos funcionalidades principales del sistema: la publicación de nuevos materiales y la búsqueda de materiales académicos.

Alcance de la funcionalidad: Publicación de materiales

Esta funcionalidad permitirá a los usuarios:

- Cargar archivos académicos (apuntes, resúmenes, etc.).
- Completar datos asociados al material (título, materia, carrera, descripción).
- Validar la información ingresada (campos obligatorios, formato y peso del archivo).
- Almacenar el material y sus metadatos en la base de datos.
- Visualizar el material publicado dentro de la plataforma.

Alcance de la funcionalidad: Búsqueda de materiales

Esta funcionalidad permitirá a los usuarios:

- Buscar materiales mediante palabras clave.
- Filtrar resultados por criterios como carrera, materia o tipo de archivo.
- Visualizar los resultados obtenidos de forma organizada.
- Acceder a la información básica del material encontrado.
- Recibir mensajes en caso de no existir coincidencias.

Este alcance incluye el análisis, diseño y desarrollo básico de dichas funcionalidades, así como la elaboración de los modelos correspondientes (casos de uso, diagramas de secuencia y contratos). No se contempla en esta etapa la implementación completa de todas las funcionalidades del sistema, sino una versión inicial que permita validar su funcionamiento.

Capítulo 2: Metodología

2.1 Ciclo de vida del proyecto

Para el desarrollo del proyecto se adopta un enfoque ágil basado en Scrum, debido a la necesidad de construir el sistema de forma incremental, permitiendo validar cada avance y realizar ajustes en función de los resultados obtenidos.

En esta etapa, el ciclo de vida se centra específicamente en las funcionalidades definidas para el avance del proyecto: **publicación de materiales académicos** y **búsqueda de materiales**.

Dado que estas funcionalidades presentan distintos niveles de complejidad, se decide abordarlas mediante iteraciones cortas o Sprints, permitiendo desarrollar, validar e integrar cada una de manera progresiva.

En una primera instancia, las iteraciones se enfocan en la funcionalidad de **publicación**, incluyendo la carga, validación y almacenamiento de materiales académicos. Esta funcionalidad constituye el eje principal del primer avance, ya que permite generar el contenido sobre el cual operará el sistema.

En una segunda etapa, se aborda la funcionalidad de **búsqueda**, la cual depende directamente de la correcta estructuración de los datos previamente almacenados. Esta incluye la implementación de filtros, consultas y visualización de resultados.

Cada Sprint contempla actividades de análisis, diseño, desarrollo y pruebas, asegurando que cada funcionalidad sea completamente operativa antes de avanzar a la siguiente.

De esta manera, el enfoque iterativo e incremental permite reducir riesgos, mejorar la calidad del sistema y garantizar una evolución coherente del proyecto.

2.2 Planificación realizada

Para llevar a la práctica el ciclo de vida definido, se realizó una planificación basada en Sprints, enfocando cada iteración en una funcionalidad específica del sistema.

Se establece un Sprint inicial de 2 semanas para el desarrollo de la funcionalidad de **publicación de materiales**, correspondiente al primer avance del proyecto. La funcionalidad de **búsqueda** será abordada en una iteración posterior, correspondiente a la segunda entrega.

Definimos que este primer Sprint tendrá una duración de 2 semanas.

2.2.1 Nuestros Recursos Humanos

El equipo se organiza de acuerdo a los roles definidos por Scrum:

- **Product Owner:** Lara, encargada de gestionar el backlog y definir prioridades.
- **Scrum Master:** Valentina, responsable de coordinar el equipo y facilitar el proceso.
- **Equipo de Desarrollo:** ambas integrantes, encargadas de la implementación, pruebas y diseño.

2.2.2 Planificación del Sprint 1 – Publicación de materiales

El objetivo de este Sprint es implementar la funcionalidad de publicación, permitiendo a los usuarios cargar materiales académicos en el sistema.

Tareas:

- Creación del formulario de carga de materiales (0,5 día)
- Implementación del mecanismo de subida de archivos (1 día)
- Validación de datos (campos obligatorios, formato, peso del archivo) (0,5 día)
- Almacenamiento del archivo y sus metadatos en la base de datos (1,5 día)
- Visualización de los materiales publicados (0,5 día)
- Pruebas y ajustes de la funcionalidad (1 día)

2.2.3 Planificación del Sprint 2 – Funcionalidad de búsqueda

La funcionalidad de búsqueda será desarrollada en una etapa posterior, e incluirá:

- Diseño de la interfaz de búsqueda
- Implementación de búsqueda por palabras clave
- Aplicación de filtros (materia, carrera, tipo de archivo)
- Visualización de resultados
- Manejo de resultados vacíos

Con esta planificación, se logra organizar el desarrollo de manera progresiva, priorizando primero la generación de contenido (publicación) y posteriormente su acceso (búsqueda), asegurando coherencia entre las funcionalidades.

2.3 Cronograma del proyecto

El desarrollo del proyecto se organiza siguiendo la metodología ágil Scrum, mediante iteraciones (Sprints) que permiten una evolución progresiva del sistema. Cada Sprint agrupa un conjunto de actividades específicas orientadas a cumplir con los objetivos de las entregas establecidas.

El cronograma se estructura en dos grandes etapas, correspondientes a la Primera Entrega y la Segunda Entrega, permitiendo avanzar desde la planificación y el diseño hasta la implementación y validación del sistema.

Tabla 1. Planificación de actividades.

Semana	Sprint	Actividades
Semana 1	Sprint 1	Revisión del proyecto de IS1, definición del ciclo de vida, planificación inicial
Semana 2	Sprint 2	Elaboración del plan de riesgos y selección de arquitectura
Semana 3	Sprint 3	Refinamiento de casos de uso y conversaciones
Semana 4	Sprint 4	Desarrollo de diagramas de secuencia y contratos de operaciones
Semana 5	Sprint 5	Desarrollo de funcionalidad básica (publicación de materiales)
Semana 6	Sprint 6	Ajustes, integración y preparación de la primera entrega
Semana 7	Sprint 7	Diseño del modelo de base de datos físico
Semana 8	Sprint 8	Desarrollo de funcionalidad principal del sistema
Semana 9	Sprint 9	Implementación de pruebas y verificación de trazabilidad
Semana 10	Sprint 10	Documentación técnica y manual de usuario

Este cronograma permite organizar el desarrollo de manera ordenada e incremental, asegurando que cada etapa aporte valor al sistema. La división en Sprints facilita el seguimiento del progreso, la detección temprana de errores y la posibilidad de realizar ajustes continuos.

Además, la planificación contempla la dependencia entre actividades, comenzando por la definición y el diseño del sistema, para luego avanzar hacia su implementación y validación.

Se utilizó la herramienta Trello para la gestión de tareas y seguimiento de los Sprints, permitiendo organizar el backlog, asignar responsabilidades y monitorear el progreso del equipo.

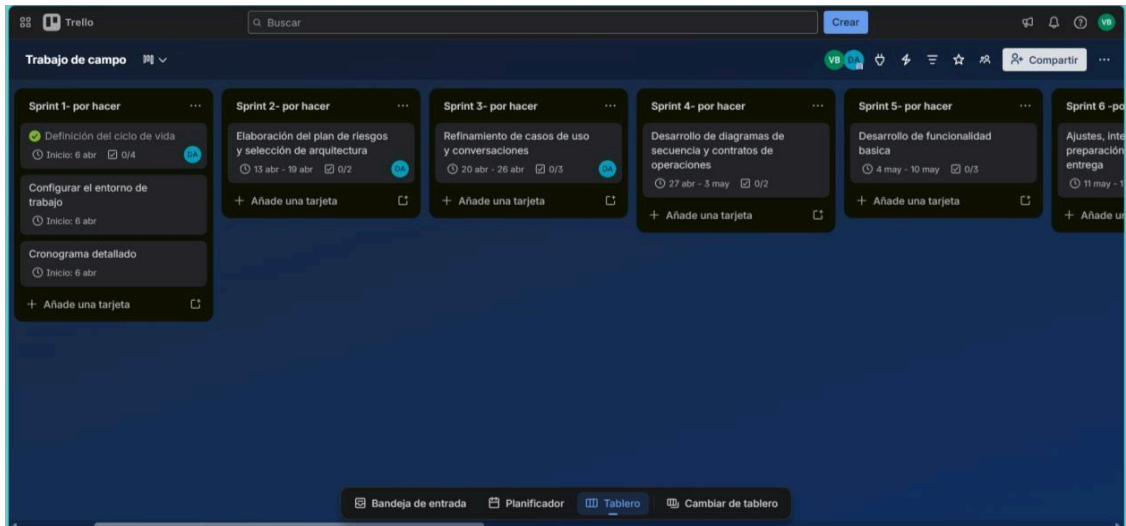


Figura 1: Organización del proyecto mediante Trello

<https://trello.com/b/WgZNBIMG/trabajo-de-campo>

2.3.1 Relación entre Sprints y funcionalidades

Los Sprints definidos en el cronograma se organizan en función de las funcionalidades principales del sistema.

En este sentido:

- Los primeros Sprints (1 a 6) corresponden a la **Primera Entrega**, enfocándose en el análisis, diseño e implementación de la funcionalidad de publicación de materiales.
- Los Sprints restantes (7 a 10) corresponden a la **Segunda Entrega**, orientados al desarrollo de la funcionalidad de búsqueda y la validación del sistema.

De esta manera, se logra mantener una planificación detallada del proyecto, alineada con el enfoque incremental definido en el ciclo de vida.

2.4 Plan de riesgos

Para asegurar que el desarrollo del proyecto no se estanque y podamos avanzar sin problemas durante este primer Sprint, armé junto con mi compañera un plan de riesgos. En él identificamos, analizamos y evaluamos las posibles amenazas que nos pueden jugar en contra. Clasificamos estos riesgos en tres frentes principales: de proyecto, de producto y de negocio.

A continuación, detallo cómo planeamos manejar los escenarios que podrían complicarnos mientras programamos el registro, el inicio de sesión, la carga de apuntes y el buscador:

2.4.1 Riesgos de Producto (La tecnología y los requerimientos)

- **Actualizaciones de herramientas:** Sabemos que las actualizaciones de las herramientas que elegimos usar nos pueden generar nuevos errores en el software. Le calculamos una probabilidad baja y un efecto serio. Nuestra estrategia para mitigarlo es revisar cuidadosamente los cambios de cada actualización para poder identificar esos errores y corregirlos.
- **Requerimientos mayores a los esperados:** Por otro lado, nos puede pasar que los requerimientos del sistema terminen siendo mayores de lo que esperamos, especialmente con la subida de archivos y los filtros. Esto tiene una probabilidad moderada y un efecto tolerable. Nuestra estrategia acá es aplicar un estricto control del alcance mediante el rol del Product Owner. Si detectamos requerimientos demasiado complejos durante el desarrollo, priorizaremos el Product Backlog y negociaremos su postergación para Sprints futuros, protegiendo así la entrega del Sprint actual.

2.4.2 Riesgos de Proyecto (Nuestro trabajo y tiempos)

- **Interfaz y usabilidad deficiente:** Como queremos que el proyecto sea súper fácil de usar para nuestros compañeros estudiantes, una interfaz deficiente sería un problema de efecto serio, aunque creemos que su probabilidad es moderada. Para controlarlo, vamos a evaluar constantemente si nuestro sistema cumple con los requisitos que pedimos, a través de pruebas de usabilidad.
- **Ausencia de un integrante del equipo:** Indisponibilidad temporal de una de las desarrolladoras debido a imprevistos personales, de salud o sobrecarga por exámenes. La probabilidad es moderada y el efecto es serio al ser un equipo de desarrollo reducido, la baja temporal reduce la capacidad operativa a la mitad, lo que impacta fuertemente en el cumplimiento de las fechas del Sprint. Para poder solucionarlo pensamos mantener el repositorio de código y la documentación sincronizados de forma diaria, registrar el progreso detallado en Trello, y asegurar un conocimiento técnico cruzado (que ambas manejemos aspectos de interfaz y base de datos) para que el desarrollo no se paralice.
- **Demoras por defectos:** Otro riesgo es que no podamos entregar el software en el tiempo esperado porque detectemos defectos a último momento. La probabilidad es baja y el efecto es tolerable. Para evitarlo pensamos implementar revisiones de código cruzadas entre las integrantes del equipo a medida que programamos.

2.4.3 Riesgos de Negocio

- **La competencia:** En el mundo real ya existen competidores (como grupos de WhatsApp o drives de la facultad), y su presencia nos puede dificultar la captación de nuevos usuarios. Esto es altamente probable y tiene un efecto serio. ¿Nuestra

estrategia? Aplicar una interfaz amigable para el usuario, optimizar nuestros procesos para acelerar el desarrollo, y hacer una diferencia clara con los productos similares que ya ofrece la competencia.

2.5 Evaluación de arquitecturas

Para el desarrollo del sistema “Univia”, se analizaron distintos estilos arquitectónicos con el objetivo de seleccionar la alternativa más adecuada según los requerimientos funcionales y no funcionales.

En primer lugar, se evaluaron arquitecturas tradicionales como **cliente-servidor** y **arquitectura en capas**, ampliamente utilizadas en aplicaciones web. Estas permiten estructurar el sistema de manera organizada, facilitando la interacción entre usuarios, el procesamiento de la lógica del sistema y la gestión de datos.

Asimismo, se consideraron arquitecturas modernas como **microservicios**, **SOA** y **cloud computing**, las cuales ofrecen alta escalabilidad y flexibilidad. Sin embargo, implican una mayor complejidad de implementación, despliegue y mantenimiento, lo cual no resulta adecuado para el alcance actual del proyecto.

Por otro lado, se analizaron arquitecturas como **tuberías y filtros**, **map-reduce**, **peer to peer**, **broker** y **publicador-suscriptor**, orientadas principalmente al procesamiento de datos o sistemas distribuidos. Estas no se ajustan al sistema propuesto, ya que “Univia” se centra en la interacción directa con usuarios y en operaciones sobre una base de datos.

En este contexto, el sistema se identifica como una **aplicación de procesamiento de transacciones**, donde los usuarios realizan operaciones de consulta y carga de información.

2.5.1 Selección de la arquitectura

A partir del análisis realizado, se selecciona una arquitectura basada en:

- **Modelo cliente-servidor**
- **Arquitectura en capas**

2.5.2 Justificación

La selección de la arquitectura se realizó teniendo en cuenta los requisitos no funcionales definidos para el sistema, priorizando la simplicidad, mantenibilidad, rendimiento y facilidad de evolución.

El modelo cliente-servidor junto con la arquitectura en capas permiten cumplir adecuadamente con estos requisitos, ya que facilitan la organización del sistema, la separación de responsabilidades y el desarrollo incremental.

En particular, la arquitectura en capas contribuye directamente al cumplimiento del **RNF09 (facilidad para incorporar nuevas funcionalidades)**, al permitir modificar o extender una capa sin afectar las demás, favoreciendo el bajo acoplamiento y la mantenibilidad.

Asimismo, esta organización facilita el desarrollo bajo metodología ágil (**RNF10**), permitiendo implementar funcionalidades de manera incremental y controlada.

Por otro lado, el modelo cliente-servidor permite gestionar múltiples usuarios concurrentes de manera centralizada, contribuyendo al cumplimiento del **RNF03 (soporte de usuarios concurrentes)** y al **RNF01 (tiempo de respuesta)**, al concentrar el procesamiento en el servidor.

Las arquitecturas alternativas fueron descartadas por no alinearse adecuadamente con los requisitos no funcionales del sistema:

- **Microservicios / SOA:**

Si bien favorecen la escalabilidad (RNF03), introducen una alta complejidad en la comunicación entre servicios, despliegue y mantenimiento, lo que dificulta el cumplimiento del **RNF09** (mantenibilidad) y del **RNF10** (desarrollo ágil) en un proyecto de alcance acotado.

- **Cloud Computing:**

No se considera una arquitectura de software sino un modelo de infraestructura. Si bien podría contribuir al cumplimiento de **RNF03** (escalabilidad), no define la organización interna del sistema ni aporta directamente a la mantenibilidad (RNF09).

- **Tuberías y filtros / Map-Reduce:**

Estas arquitecturas están orientadas al procesamiento intensivo de datos en flujo, lo cual no se corresponde con los requerimientos del sistema. Además, no favorecen el cumplimiento del **RNF01** (tiempo de respuesta interactivo) ni del **RNF04** (interfaz amigable), ya que no están diseñadas para interacción directa con usuarios.

- **Peer to Peer:**

No permite un control centralizado de los datos, lo que dificulta el cumplimiento de requisitos como **RNF05** (seguridad de datos) y **RNF11** (respaldo y recuperación).

- **Broker / Publicador–Suscriptor:**

Están orientadas a sistemas distribuidos y basados en eventos. Su uso introduciría complejidad innecesaria, afectando la mantenibilidad (**RNF09**) y el desarrollo ágil (**RNF10**), sin aportar beneficios significativos al tipo de sistema propuesto.

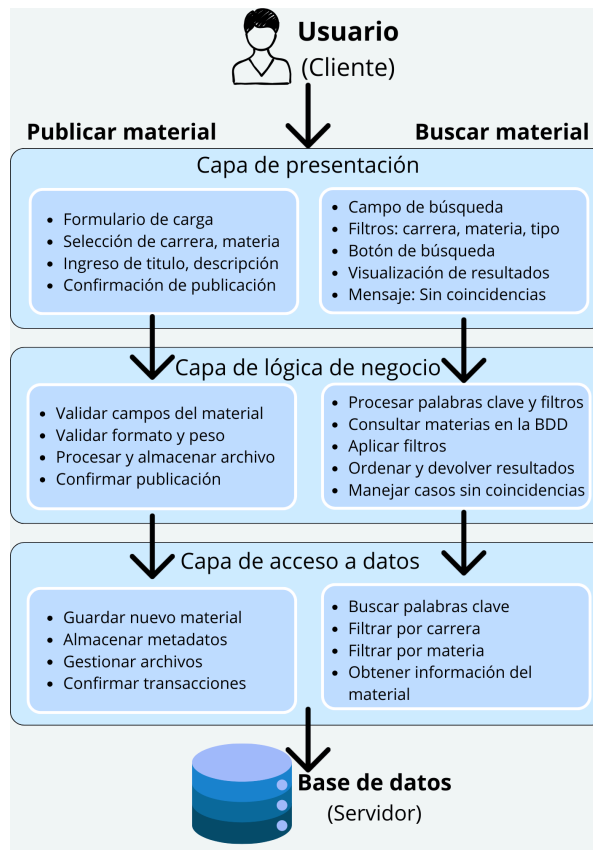


Figura 2. Diagrama de arquitectura

La arquitectura representada corresponde a un modelo cliente-servidor organizado en capas, donde el cliente interactúa con la capa de presentación, la cual se comunica con la lógica de negocio y, finalmente, con la base de datos.

2.6 Diagrama de casos de uso

El diagrama de casos de uso fue refinado en función del alcance definido para el presente avance, el cual se centra en las funcionalidades de publicación y búsqueda de materiales académicos.

Se eliminaron aquellas funcionalidades que no forman parte de la implementación actual, como registro, inicio de sesión, valoración y administración, con el objetivo de enfocar el modelo en las interacciones principales del sistema.

De esta manera, el diagrama resultante representa las operaciones clave que serán desarrolladas, permitiendo una mejor comprensión del comportamiento del sistema en esta etapa.

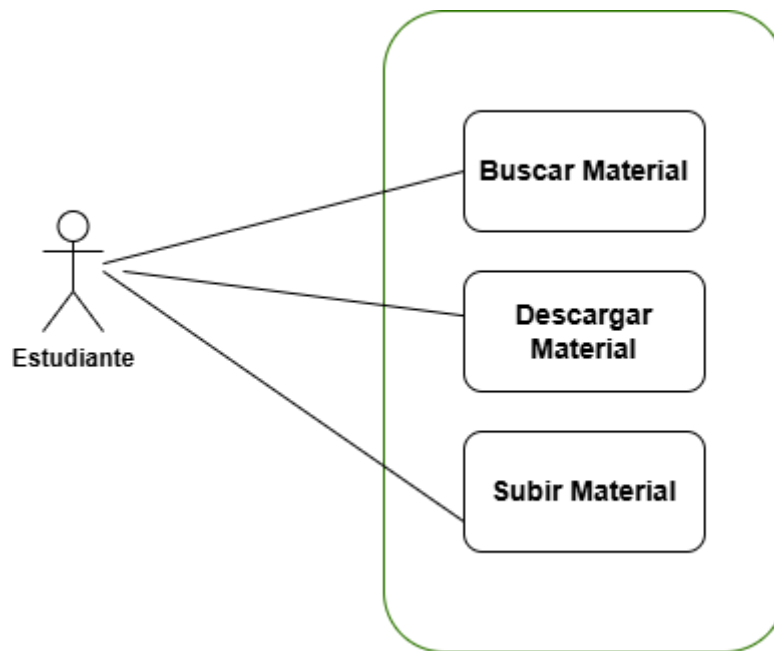


Figura 3: Diagrama de casos de uso

2.7 Conversaciones

Con el objetivo de representar el comportamiento del sistema y la interacción con los usuarios, se seleccionaron tres conversaciones correspondientes a funcionalidades clave: búsqueda de material, obtención de material y publicación de nuevos materiales.

Estas conversaciones permiten describir distintos escenarios de uso del sistema, incluyendo la consulta de información, el acceso a los recursos disponibles y la incorporación de nuevos contenidos por parte de los usuarios.

A través de estos casos, se modelan tanto los flujos principales como las interacciones más relevantes, sirviendo como base para la elaboración de los diagramas de secuencia y el análisis del funcionamiento del sistema.

♦ Caso de uso: Buscar material

Actor: Estudiante

Visión general: El usuario ingresa a la plataforma y utiliza el buscador para encontrar material académico (apuntes, libros, exámenes, etc.) mediante filtros como nombre, carrera, materia, tipo de archivo o universidad. El sistema muestra los resultados coincidentes.

Precondición 1: El usuario está autenticado y accede a la funcionalidad de búsqueda.

Postcondición 1: El sistema muestra los resultados correspondientes según los criterios ingresados.

Precondición 2: El usuario realiza una búsqueda sin resultados válidos.

Postcondición 2: El sistema muestra un mensaje indicando que no se encontraron coincidencias.

Tabla 2. Caso de uso: Buscar material.

Acción	Curso normal	Curso alternativo 1	Curso alternativo 2
1. El usuario accede a la opción “Buscar material” e ingresa criterios (ej.: materia, carrera, palabra clave, tipo de archivo)			
2. El sistema recibe los parámetros de búsqueda	2.1 El sistema consulta en la base de datos los materiales coincidentes	2.2 El sistema no encuentra resultados 2.2.1 Muestra mensaje “No se encontraron coincidencias”	2.3 El sistema detecta campos vacíos o inválidos 2.3.1 Solicita corrección y vuelve al paso 1
3. El sistema muestra lista de resultados disponibles con nombre, descripción, tipo y usuario que lo subió			
4. Fin del caso de uso			

♦ Caso de uso: Descargar material

Actor: Estudiante

Visión general: El usuario, luego de buscar y seleccionar un material, puede obtenerlo descargándolo en formato digital o contactando al dueño del archivo si se ofrece en formato físico o bajo condiciones específicas (gratuito o pago).

Precondición 1: El usuario está logueado y ha seleccionado un material disponible.

Postcondición 1: El sistema permite la descarga o muestra el método de contacto con el propietario.

Precondición 2: El usuario intenta acceder a un archivo restringido o eliminado.

Postcondición 2: El sistema informa que el material no está disponible.

Tabla 3. Caso de uso: Descargar material.

Acción	Curso normal	Curso alternativo 1	Curso alternativo 2
1. El usuario selecciona un material desde los resultados de búsqueda			
2. El sistema muestra los detalles del archivo (nombre, descripción, tipo, tamaño, condiciones)	2.1 El sistema ofrece opción de descarga directa (si es gratuito o permitido)	2.2 El sistema indica que el material debe ser solicitado al usuario propietario 2.2.1 Muestra botón de contacto o formulario	2.3 El material fue eliminado o restringido 2.3.1 Muestra mensaje de error y vuelve al paso 1
3. El usuario descarga el archivo y ese se guarda en su dispositivo			
4. Fin del caso de uso			

♦ Caso de uso: Subir un nuevo material.

Actor: Estudiante

Visión general: El estudiante, luego de iniciar sesión, accede a la funcionalidad para subir material académico. Completa un formulario con los datos requeridos (título, materia, descripción, tipo de archivo y modalidad) y selecciona un archivo desde su dispositivo. El sistema valida los datos ingresados y el archivo (tamaño y formato). Si las

validaciones son correctas, el sistema guarda el archivo, crea la publicación y la deja disponible para otros usuarios.

Precondición 1: El usuario selecciona un material y completa todos los campos requeridos correctamente.

Postcondición 1: El material académico queda publicado en la plataforma y está disponible para que otros usuarios puedan verlo según las condiciones definidas (gratuito o pago).

Precondición 2: El usuario intenta subir un archivo que excede el tamaño permitido o tiene un formato no compatible.

Postcondición 2: El sistema no publica el material, y muestra un mensaje de error indicando la causa (el tamaño o formato no válido) y el usuario debe corregir el archivo antes de volver a intentarlo.

Tabla 4. Caso de uso: Subir un nuevo material.

Acción	Curso normal	Curso alternativo 1	Curso alternativo 2
1. El usuario selecciona un archivo y completa el formulario.			
2. El sistema verifica los datos ingresados.	2.1 El sistema valida el tamaño y el formato del archivo	2.2 El sistema detecta que el archivo excede el tamaño permitido 2.2.1 Se muestra un mensaje de error indicando el límite aceptado 2.2.2 Puede elegir otro archivo o ajustar el tamaño y volver a intentarlo.	2.3 El sistema detecta que el formato no es aceptado 2.3.1 Se muestra un mensaje de error indicando los archivos admitidos 2.3.2 Puede modificar el formato y volverlo a cargar
3. El sistema guarda el archivo			
4. El sistema crea la publicación con los datos y el archivo.			

5. El sistema guarda la publicación.	5.1 La publicación se almacena correctamente.		
6. El sistema muestra un mensaje de éxito	6.1 "Publicación subida con éxito".		
7. Fin del caso de uso			

2.8 Diagramas de Secuencia

Se elaboraron diagramas de secuencia para los casos de uso más representativos, como "Buscar material", "Obtener material" y "Subir material". Estos diagramas muestran cómo se comunican los objetos del sistema (usuario, interfaz, controlador, base de datos) a lo largo del tiempo para cumplir una funcionalidad.

Se utilizó un enfoque típico MVC (Modelo-Vista-Controlador), en donde:

- El usuario interactúa con la interfaz gráfica.
- La interfaz envía las solicitudes al controlador.
- El controlador valida y consulta/actualiza los datos en la base de datos.
- Se devuelve una respuesta al usuario.

Esto refleja la arquitectura planteada y la lógica de interacción propuesta.

Caso de uso: Buscar Material

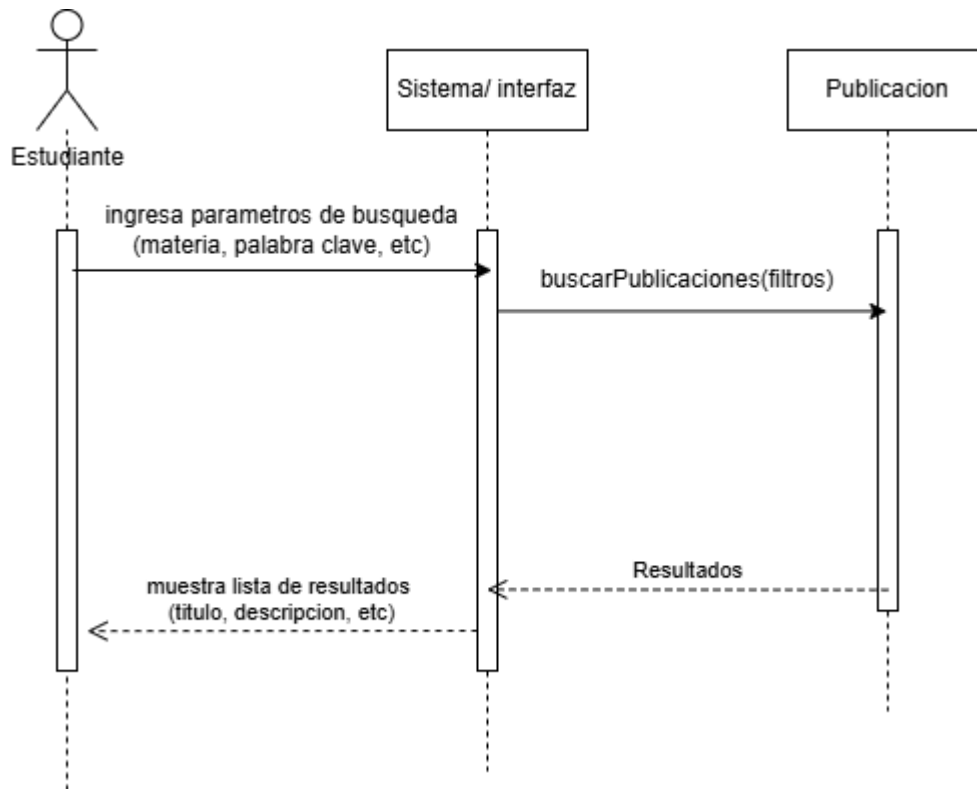


Figura 4. Diagrama de secuencia: Buscar Material

Caso de uso: Descargar Material

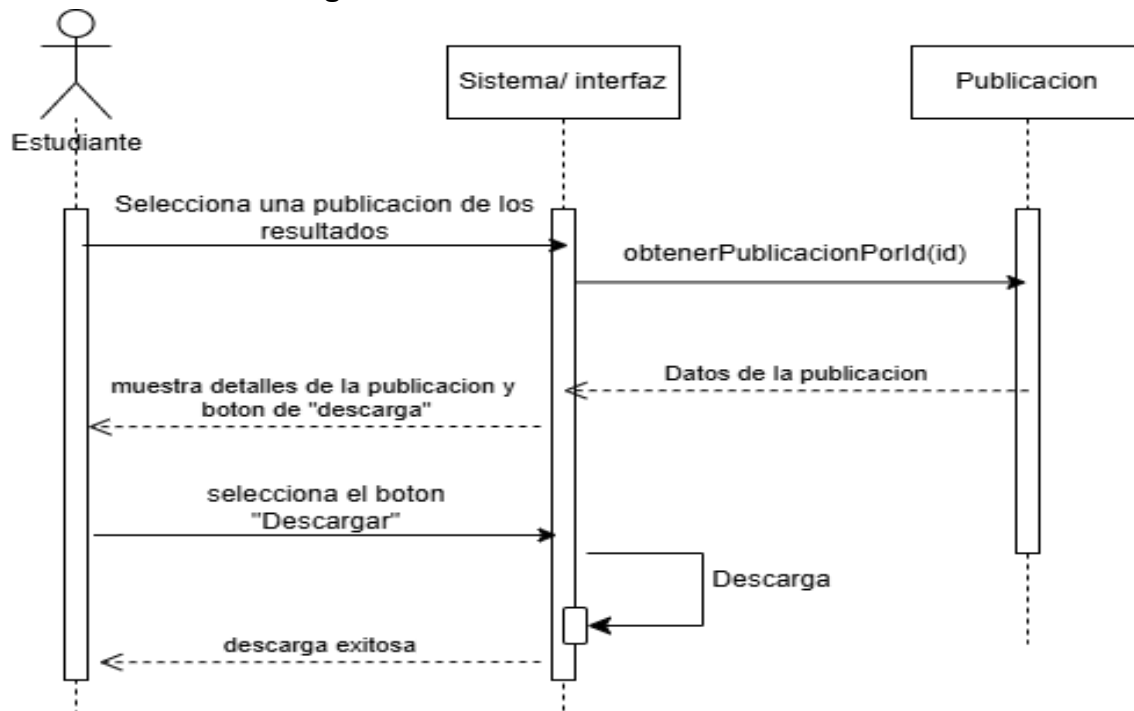


Figura 5. Diagrama de secuencia: Descargar Material

Caso de uso: Subir Material

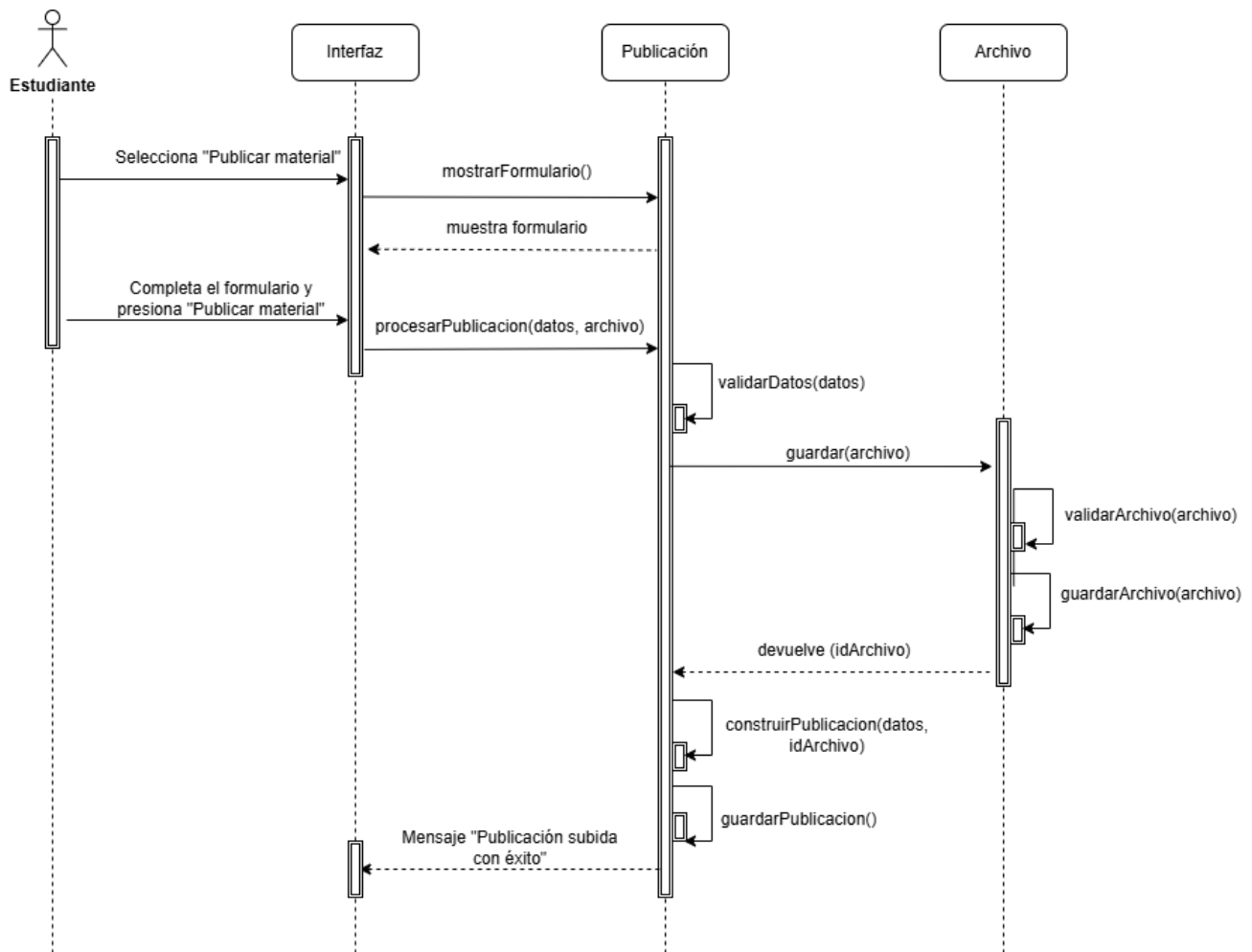


Figura 6. Diagrama de secuencia: Subir Material

2.9 Contrato de operaciones críticas

En el presente proyecto, los contratos de operaciones describen el comportamiento de las funcionalidades críticas seleccionadas: publicación y búsqueda de materiales académicos.

Estos contratos especifican las condiciones necesarias para ejecutar cada operación (precondiciones) y los resultados que se obtienen luego de su ejecución (postcondiciones), permitiendo definir de manera clara cómo el sistema gestiona la carga y consulta de materiales dentro de la plataforma.

2.9.1 Contrato: Guardar publicación

Tabla 5. Contrato de operaciones críticas C.U. Subir un nuevo material.

1. Nombre	guardarPublicacion(datos, archivo)
2. Referencia Cruzada	Caso de uso: Publicar material
3. Responsabilidades	Permitir al usuario registrar una nueva publicación en la plataforma, validando los datos ingresados, gestionando la carga del archivo y almacenando la información asociada en el sistema.
4. Excepciones	- Si los datos de la publicación no son válidos, la operación no se realiza. - Si el archivo no es válido, la operación no se realiza. - Si ocurre un error durante la carga o almacenamiento, el sistema informa al usuario.
5. Pre-condiciones	- EL usuario se encuentra autenticado en el sistema. - El usuario ha completado los datos requeridos para la publicación. - El usuario ha seleccionado un archivo para subir.
6. Post-condiciones	- Los datos de la publicación son validados correctamente. - El archivo es validado y almacenado en el servidor. - Se genera un identificador único para el archivo (id-archivo) - Se registra el archivo en la base de datos. - Se crea una nueva publicación asociada al archivo mediante el id_archivo. - La publicación queda disponible en el sistema.

2.9.2 Contrato: Buscar material

Tabla 6. Contrato de operaciones críticas C.U. Buscar material.

1. Nombre	buscarMaterial(palabrasClave, filtros: materia, carrera, tipo de archivo)
2. Referencia Cruzada	Caso de uso: Buscar material
3. Responsabilidades	Permitir al usuario buscar materiales académicos mediante palabras clave y/o filtros, procesando la consulta y devolviendo los resultados correspondientes.
4. Excepciones	- Si no se encuentran coincidencias, el sistema informa al usuario mediante un mensaje adecuado. - Si los criterios de búsqueda son inválidos o vacíos, el sistema solicita su corrección.
5. Pre-condiciones	- El usuario accede a la interfaz de búsqueda.

	- El sistema dispone de materiales almacenados en la base de datos.
6. Post-condiciones	- Se realiza una consulta en la base de datos en función de los criterios ingresados. - Se obtiene un conjunto de materiales que coinciden con la búsqueda. - Los resultados son mostrados al usuario de forma organizada.

2.10 Diagrama de entidad relación

El diagrama de entidad-relación representa la estructura de la base de datos del sistema y las relaciones existentes entre sus entidades principales. A través de este modelo identificamos los datos necesarios para el funcionamiento de la plataforma, como usuarios, publicaciones, archivos, materias, carreras y universidades, así como sus respectivas vinculaciones. Este diseño permite organizar la información de manera eficiente, garantizando integridad, consistencia y una correcta gestión de los datos dentro del sistema.

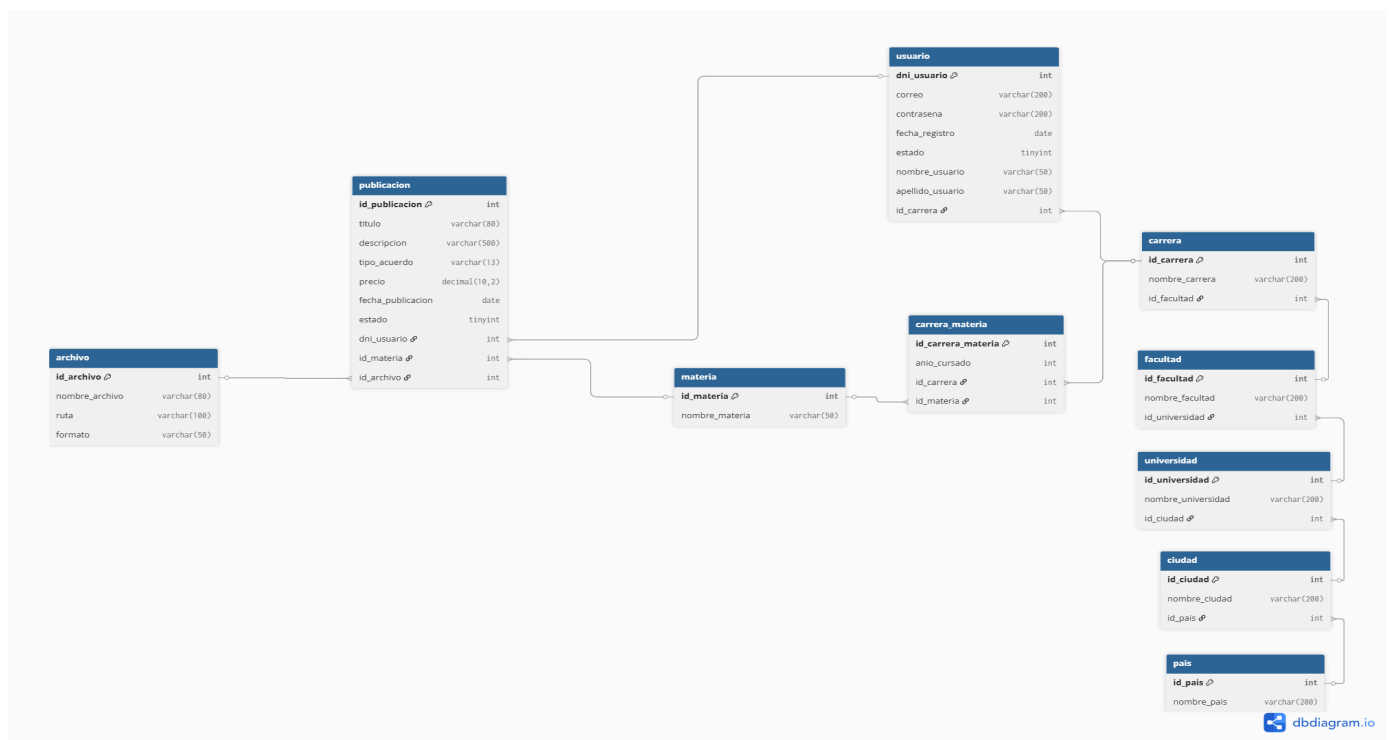


Figura 7. Diagrama de entidad relación

2.11 Diccionario de Datos

El diccionario de datos define los atributos principales de cada entidad del sistema. Para cada atributo se especificó:

- Nombre del campo.
- Tipo de dato (texto, entero, fecha, etc.).
- Descripción funcional.
- Restricciones (por ejemplo, campo obligatorio, único, clave primaria).

Tabla: Pais

Módulo: Geografico

Descripción: Esta tabla se diseñó para registrar los países a los que pertenecen las distintas universidades.

Características de los datos:

Tabla 7. Diccionario: Pais

Campo	Tipo	Long	Significado
id_pais	int	11	Contiene una identificación única del Pais
nombre_pais	varchar	200	Contiene el nombre del Pais

Restricciones:

Campo	Tipo de Restricción
id_pais	Primary Key

Tabla: Ciudad

Módulo: Geografico

Descripción: Esta tabla se diseñó para registrar las ciudades de cada País.

Características de los datos:

Tabla 8. Diccionario: Ciudad

Campo	Tipo	Long	Significado
id_ciudad	int	11	Contiene una identificación única de la ciudad
nombre_ciudad	varchar	200	Contiene el nombre de la ciudad

id_pais	int	11	Contiene una identificación única del Pais
---------	-----	----	--

Restricciones:

Campo	Tipo de Restricción
id_ciudad	Primary Key

Claves foráneas:

Campo	Entidad Asociada
id_pais	Pais

Tabla: facultad

Módulo: institucional

Descripción: Esta tabla se diseñó para almacenar los datos de la facultad que pertenece a una universidad

Características de los datos:

Tabla 9. Diccionario: Facultad

Campo	Tipo	Long	Significado
id_facultad	int	11	Contiene una identificación única de la facultad
Nombre_facultad	Varchar	80	Contiene el nombre de la facultad
id_universidad	int	11	Contiene la identificación de la universidad a la que pertenece la facultad

Restricciones:

Campo	Tipo de Restricción
id_facultad	Primary Key

Claves foráneas:

Campo	Entidad Asociada
id_universidad	Universidad

Tabla: Universidad

Módulo: Institucional

Descripción: Esta tabla se diseñó para almacenar los datos de la universidad.

Características de los datos:

Tabla 10. Diccionario: Universidad

Campo	Tipo	Long	Significado
id_universidad	int	11	Contiene una identificación única de la universidad
nombre_universidad	varchar	200	Contiene el nombre de la universidad
Id_ciudad	int	11	Contiene la ciudad donde se encuentra la universidad

Restricciones:

Campo	Tipo de Restricción
id_universidad	Primary Key

Claves foráneas:

Campo	Entidad Asociada
id_ciudad	Ciudad

Tabla: Carrera

Módulo: Estructura académica

Descripción: Contiene información sobre las carreras que ofrece la universidad

Características de los datos:

Tabla 11. Diccionario: Carrera

Campo	Tipo	Long	Significado
-------	------	------	-------------

id_carrera	int	11	Contiene una identificación única de la carrera
nombre_carrera	varchar	200	Contiene el nombre de la carrera
id_facultad	int	11	Contiene la identificación única de la facultad

Restricciones:

Campo	Tipo de Restricción
id_carrera	Primary Key

Claves foráneas:

Campo	Entidad Asociada
id_facultad	Facultad

Tabla: Carrera_materia

Módulo: Estructura academica

Descripción: es una tabla intermedia entre carrera y materia

Características de los datos:

Tabla 12. Diccionario: Carrera_materia

Campo	Tipo	Long	Significado
id_carrera_materia	int	11	contiene una identificación única
anio_cursado	date	-	contiene el año en el que se dicta la materia
id_carrera	Int	11	Contiene la identificación de la carrera
id_materia	Int	11	Contiene la identificación de la materia

Restricciones:

Campo	Tipo de restricción
id_carrera_materia	Primary key

Claves foráneas:

Campo	Entidad asociada
id_carrera	Carrera
Id_materia	Materia

Tabla: Materia

Módulo: Estructura académica

Descripción: Esta tabla se diseñó para almacenar los datos de las materias de cada carrera

Características de los datos:

Tabla 13. Diccionario: Materia

Campo	Tipo	Long	Significado
id_materia	int	11	Contiene una identificación única de la materia
nombre_materia	varchar	50	Contiene el nombre de la materia

Restricciones:

Campo	Tipo de Restricción
id_materia	Primary Key

Tabla: Usuario

Módulo: gestión de usuario

Descripción: Esta tabla se diseñó para almacenar los datos del usuario del sistema.

Características de los datos:

Tabla 14. Diccionario: Usuario

Campo	Tipo	Long	Significado
dni_usuario	int	8	Contiene una identificación única del usuario
correo	varchar	200	Contiene el correo electrónico del usuario
contraseña	varchar	200	Contiene la contraseña del usuario
fecha_registro	date	-	Contiene la fecha de registro del usuario en el sistema
estado	tinyint	1	Contiene el estado del usuario, activo(1) o desactivado(0)
nombre_usuario	Varchar	50	Contiene el nombre del usuario
apellido_usuario	Varchar	50	Contiene el apellido del usuario

Restricciones:

Campo	Tipo de Restricción
dni_usuario	Primary Key
correo	Unique

Tabla: Publicación

Módulo: publicación y recursos

Descripción: Esta tabla se diseñó para almacenar los datos de la publicación hecha por el usuario

Características de los datos:

Tabla 15. Diccionario: Publicación

Campo	Tipo	Long	Significado
id_publicacion	int	11	Contiene una identificación única de la publicación
titulo	Varachar	80	Contiene el título de la publicación
descripcion	Varchar	500	Contiene la descripción de la publicación
tipo_acuerdo	Varchar	13	Contiene el tipo de acuerdo (gratis o pago)
precio	float	100	Contiene el precio del material publicado en el caso de que sea pago
fecha_publicacion	date	-	Contiene la fecha en la que se realizó la publicación
estado	tinyint	1	Contiene el estado de la publicación puede ser activo(1) o inactivo(0)
dni_usuario	int	8	Contiene la identificación del usuario que hizo la publicación
id_materia	int	11	Contiene la identificación de la materia a la que corresponde a la publicación
id_archivo	int	11	contiene la identificación del archivo que sera publicado

Restricciones:

Campo	Tipo de Restricción
id_publicacion	Primary Key

Claves foráneas:

Campo	Entidad Asociada
dni_usuario	Usuario
id_materia	Materia
id_archivo	Archivo

Tabla: Archivo

Módulo: publicacion y recursos

Descripción: Esta tabla se diseñó para almacenar los archivos que se publican.

Características de los datos:

Tabla 16. Diccionario: Archivo

Campo	Tipo	Long	Significado
id_archivo	int	11	Contiene una identificación única del archivo
nombre_archivo	Varchar	80	Contiene el nombre original del archivo
ruta	Varchar	100	Contiene el la ruta del archivo
formato	Varchar	50	Contiene la extensión del archivo (.PDF, .zip, etc)

Restricciones:

Campo	Tipo de Restricción
id_archivo	Primary Key

Capítulo 3: Herramientas

3.1 Herramientas utilizadas

Para el desarrollo del proyecto se utilizaron diversas herramientas que permitieron organizar, diseñar e implementar el sistema de manera eficiente, en concordancia con la metodología ágil adoptada.

- Trello: como herramienta de gestión de proyectos, permitiendo organizar las tareas en función de los Sprints, asignar responsabilidades y realizar el seguimiento del progreso.
- Canva y [Draw.io](https://draw.io): utilizadas para la elaboración de diagramas, tales como el diagrama de arquitectura, casos de uso y diagramas de secuencia, facilitando la representación visual de las funcionalidades del sistema.
- Visual Studio Code: como entorno de desarrollo que permitió la implementación y organización del código fuente, utilizando tecnologías como PHP y CSS.
- GitHub: como sistema de control de versiones, utilizado para la gestión y colaboración del código fuente durante la realización del programa.
- XAMPP: como entorno de servidor local, facilitando la ejecución y prueba del sistema durante su desarrollo.
- Google Docs / Word: para la redacción colaborativa de los capítulos del informe y organización general del trabajo.

En conjunto, estas herramientas permitieron llevar adelante el proyecto de manera organizada, favoreciendo el trabajo colaborativo, la correcta documentación y el desarrollo progresivo de las funcionalidades seleccionadas.

3.2 Bibliografía consultada

- [\[1\] R. S. Pressman, *Ingeniería de Software: Un enfoque práctico*, McGraw-Hill, 2010.](#)
- [\[2\] I. Sommerville, *Ingeniería de Software*, Pearson Educación, 2011.](#)
- [\[3\] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Patrones de Diseño*, Addison-Wesley, 1995.](#)
- [\[4\] Scrum Guides, "The Scrum Guide". Disponible en: <https://scrumguides.org>](#)
- [\[5\] Trello, "Trello". Disponible en: <https://trello.com>](#)
- [\[6\] Draw.io \(diagrams.net\), "Diagrams.net". Disponible en: <https://www.diagrams.net>](#)
- [\[7\] W3Schools, "Web Development Tutorials". Disponible en: <https://www.w3schools.com>](#)

3.3 Autoevaluación del Primer Avance

En cumplimiento con la rúbrica de evaluación de la asignatura, nuestro grupo considera que los entregables alcanzan los criterios de la categoría "Implementado" en los siguientes aspectos:

- **Procesos de Software:** Se evaluó y seleccionó una arquitectura ClienteServidor en capas, justificando su adecuación para el paso a la implementación.
- **Gestión de Proyectos:** Se planificaron los Sprints y se estimaron los tiempos utilizando el método Scrum.
- **Gestión de Riesgos:** Se revisaron los riesgos de la etapa de análisis y se documentaron los nuevos enfocados en el desarrollo.
- **Comunicación:** Se mantiene la trazabilidad del proyecto utilizando terminología técnica adecuada a la ingeniería de software.

Anexos

Anexo A: Enlace al repositorio git

<https://github.com/Lara141/univia.git>

Anexo B: Diagrama de secuencia anterior

Este diagrama de secuencia es el que fue presentado en la primera entrega del trabajo.

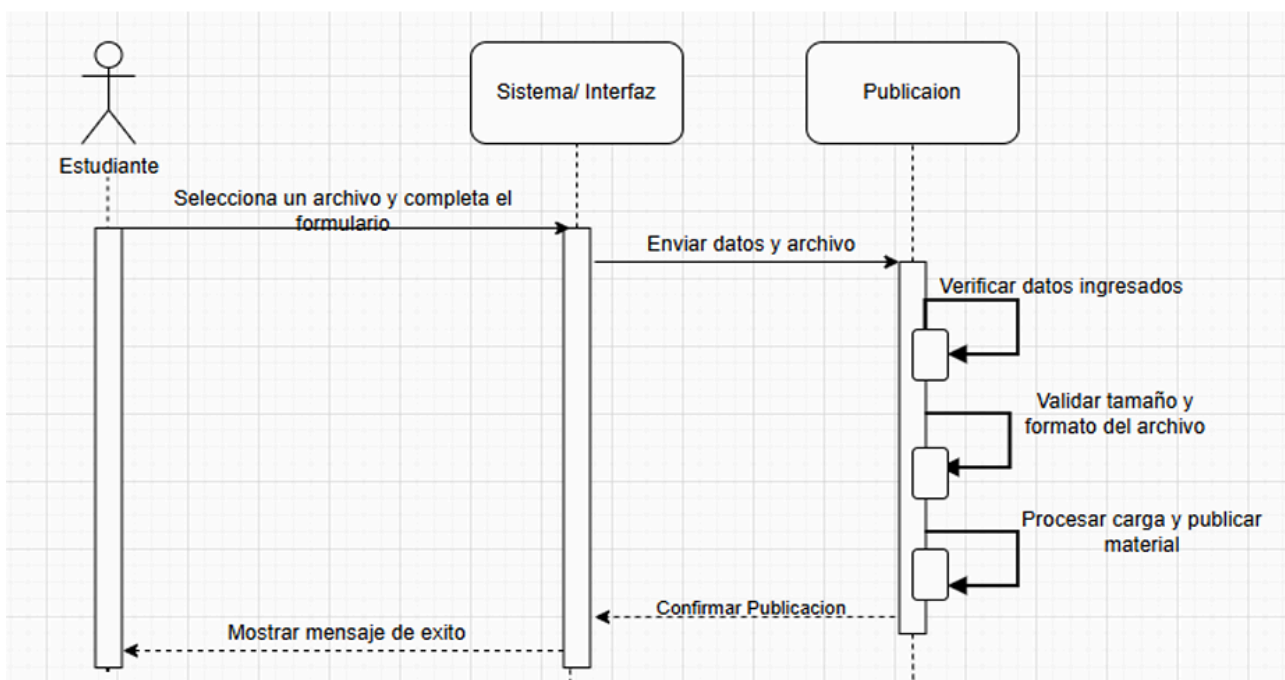


Figura 8. Diagrama de secuencia anterior

Anexo C: Código fuente actualizado

- PublicacionController.php

```
public function mostrarFormulario()
{
    if (!$this->usuarioLogueado()) {
        return redirect()->to('/');
```

```

    }

    return view('formulario_publicacion', [

        'usuario' => session()->get('usuario'),

    ]);
}

public function guardar()
{
    if (!$this->usuarioLogueado()) {

        return redirect()->to('/');

    }

    try {

        $datos = [

            'titulo' => $this->request->getPost('titulo'),

            'descripcion' => $this->request->getPost('descripcion'),

            'materia' => $this->request->getPost('materia'),

            'tipo' => $this->request->getPost('tipo_recurso'),

            'tipo_acuerdo' => $this->request->getPost('tipo_acuerdo'),

            'precio' => $this->request->getPost('precio'),

            'dni' => session()->get('usuario')['dni_usuario'],

        ];

        $archivo = $this->request->getFile('archivo');

        //Aqui llamamos a procesarPublicacion

        $this->publicacionService->procesarPublicacion($datos, $archivo);

        return redirect()->to('publicaciones/propias')->with('mensaje', 'Publicación subida con
        éxito');

    } catch (\Exception $e) {

```

```
        return redirect()->back()->withInput()->with('error', $e->getMessage());
    }
}
```

- ApiController.php

```
class ApiController extends BaseController
{

    public function materias()
    {
        $materias = $this->materiaService->obtenerMaterias();
        return $this->response->setJSON([
            'success' => true,
            'data' => $materias,
        ]);
    }

    public function tipos()
    {
        $tipos = [
            ['id' => 1, 'nombre' => 'Apunte'],
            ['id' => 2, 'nombre' => 'Resumen'],
            ['id' => 3, 'nombre' => 'Examen']
        ];
        return $this->response->setJSON([
            'success' => true,
```

```

        'data' => $tipos,
    ]);
}

public function acuerdos()
{
    $acuerdos = [
        ['id' => 1, 'nombre' => 'Gratis'],
        ['id' => 2, 'nombre' => 'Pago']
    ];

    return $this->response->setJSON([
        'success' => true,
        'data' => $acuerdos,
    ]);
}

public function publicacion($id)
{
    $publicacion = $this->publicacionService->obtenerPublicacionPorId((int) $id);

    if (!$publicacion) {
        return $this->response->setJSON([
            'success' => false,
            'message' => 'Publicación no encontrada',
        ]->setStatusCode(404);
    }

    return $this->response->setJSON([
        'success' => true,
        'data' => $publicacion,
    ]);
}

```

```

    });
}

public function publicacionesUsuario()
{
    if (!$session()->get('isLoggedIn')) {
        return $this->response->setJSON([
            'success' => false,
            'message' => 'Usuario no autenticado',
        ]->setStatusCode(401);
    }

    $dni = session()->get('usuario')['dni_usuario'];

    $publicaciones = $this->publicacionService->obtenerPublicacionesUsuario($dni);

    return $this->response->setJSON([
        'success' => true,
        'data' => $publicaciones,
        'count' => count($publicaciones),
    ]);
}

public function porMateria($idMateria)
{
    $publicaciones = $this->publicacionService->obtenerPublicacionesPorMateria((int)
    $idMateria);

    return $this->response->setJSON([
        'success' => true,
        'data' => $publicaciones,
        'count' => count($publicaciones),
    ]);}}

```

- PublicacionModel.php (en la carpeta Model)

```

class PublicacionModel extends Model {

    public function obtenerPublicacionesUsuario(string $dni, bool $soloActivas = true): array

    {

        $builder = $this->builder();

        $builder->select('p.*, m.nombre_materia, a.nombre_archivo as file_name, a.ruta');

        $builder->from('publicacion p');

        $builder->join('materia m', 'm.id_materia = p.id_materia', 'left');

        $builder->join('archivo a', 'a.id_archivo = p.id_archivo', 'left');

        $builder->where('p.dni_usuario', $dni);

        if ($soloActivas) {

            $builder->where('p.estado', 1);

        }

        $builder->orderBy('p.fecha_publicacion', 'DESC');

        return $builder->get()->getResultArray();

    }

}

```

- PublicacionService.php

```

class PublicacionService

{

    public function procesarPublicacion(array $datos, $archivo)

```

```

{
    $this->validarDatos($datos);

    if (!$archivo || !$archivo->isValid()) {
        throw new \Exception("El archivo es obligatorio para crear una publicación");
    }

    $idArchivo = $this->archivoService->guardar($archivo);

    $publicacion = $this->construirPublicacion($datos, $idArchivo);

    $this->guardarPublicacion($publicacion);
}

```

```

public function procesarArchivo($archivo)

```

```

{
    return $this->archivoService->guardar($archivo);
}

```

```

public function obtenerPublicacionesUsuario(string $dni, bool $soloActivas = true): array

```

```

{
    $builder = $this->publicacionModel->builder();

```

```

    $builder->select('publicacion.*', m.nombre_materia, a.nombre_archivo as file_name, a.ruta');

```

```

    $builder->join('materia m', 'm.id_materia = publicacion.id_materia', 'left');

```

```

    $builder->join('archivo a', 'a.id_archivo = publicacion.id_archivo', 'left');

```

```

    $builder->where('publicacion.dni_usuario', $dni);

```

```

    if ($soloActivas) {

```

```

        $builder->where('publicacion.estado', 1);

```

```

    }

```

```

    $builder->orderBy('publicacion.fecha_publicacion', 'DESC');

```

```

        return $builder->get()->getResultArray();
    }

    public function obtenerPublicacionPorId(int $id): ?array
    {
        $builder = $this->publicacionModel->builder();

        $builder->select('publicacion.*, a.nombre_archivo as file_name, a.ruta');
        $builder->join('archivo a', 'a.id_archivo = publicacion.id_archivo', 'left');
        $builder->where('publicacion.id_publicacion', $id);

        return $builder->get()->getRowArray();
    }

    public function obtenerPublicacionesPorMateria(int $idMateria): array
    {
        $builder = $this->publicacionModel->builder();

        $builder->select('publicacion.*, m.nombre_materia, a.nombre_archivo as file_name, a.ruta');
        $builder->join('materia m', 'm.id_materia = publicacion.id_materia', 'left');
        $builder->join('archivo a', 'a.id_archivo = publicacion.id_archivo', 'left');
        $builder->where('publicacion.id_materia', $idMateria);
        $builder->where('publicacion.estado', 1);
        $builder->orderBy('publicacion.fecha_publicacion', 'DESC');

        return $builder->get()->getResultArray();
    }

    private function validarDatos(array $datos)
    {
        $camposObligatorios = ['titulo', 'descripcion', 'materia', 'tipo', 'tipo_acuerdo', 'dni'];

        foreach ($camposObligatorios as $campo) {
            if (empty($datos[$campo])) {

```

```

        throw new \Exception("El campo '{$campo}' es obligatorio");
    }

    if ($datos['tipo_acuerdo'] === 'pago' && empty($datos['precio'])) {
        throw new \Exception("El precio es obligatorio para publicaciones de pago");
    }

    $tiposAcuerdoValidos = ['gratis', 'pago'];

    if (!in_array($datos['tipo_acuerdo'], $tiposAcuerdoValidos)) {
        throw new \Exception("El tipo de acuerdo seleccionado no es válido");
    }

    if (strlen($datos['titulo']) < 3) {
        throw new \Exception("El título debe tener al menos 3 caracteres");
    } }

private function guardarPublicacion(array $data)
{
    $resultado = $this->publicacionModel->insert($data);

    if (!$resultado) {
        throw new \Exception("Error al guardar la publicación en la base de datos");
    }
}
}

```

- MateriaService.php

```

class MateriaService
{
    public function obtenerMaterias(): array {

```

```

$db = Database::connect();

return $db->table('materia')->orderBy('nombre_materia', 'ASC')->get()->getResultArray();
}
}

```

- ArchivoService.php

```

class ArchivoService
{
    public function guardar($file)
    {
        if (!$file || !$file->isValid()) {
            throw new \Exception('Archivo inválido o no proporcionado');
        }

        $this->validarArchivo($file);

        return $this->guardarArchivo($file);
    }

    private function validarArchivo($file): void
    {
        if ($file->getSize() > self::MAX_TAMANIO) {
            throw new \Exception('El archivo excede el tamaño máximo permitido (20MB)');
        }

        $extension = strtolower($file->getClientExtension());

        if (!in_array($extension, self::EXTENSIONES_PERMITIDAS)) {
            throw new \Exception('Tipo de archivo no permitido. Extensiones válidas: ' .
                implode(', ', self::EXTENSIONES_PERMITIDAS));
        }
    }
}

```

```

    }

    private function guardarArchivo($file): int
    {
        $extension = strtolower($file->getClientExtension());

        $nombre = $file->getRandomName();

        if (!$file->move(self::RUTA_UPLOADS, $nombre)) {
            throw new \Exception('Error al guardar el archivo en el servidor');
        }

        $idArchivo = $this->archivoModel->insert([
            'nombre_archivo' => $file->getClientName(),
            'ruta' => self::RUTA_UPLOADS . '/' . $nombre,
            'formato' => $extension,
        ]);

        if (!$idArchivo) {
            throw new \Exception('Error al guardar la información del archivo en la base de
datos');
        }

        return $idArchivo;
    }

    public function obtener($idArchivo)
    {
        return $this->archivoModel->find($idArchivo);
    }
}

```